Benefits & Risks of Microservice Architecture

University ERP · Advanced Software Engineering · UCSC

1. Introduction

This document evaluates the decision to adopt a microservice architecture for the University ERP system. The system replaces a hypothetical legacy monolith that handled student records, course management, examination, results, transcripts, authentication, notifications, and reporting as a single deployable unit. The analysis covers both the expected benefits and the known risks, together with mitigations applied in this design.

2. Benefits

2.1 Independent Deployability

Each of the eight services (Auth, Student, Course, Exam, Result, Transcript, Notification, Reporting) can be built, tested, and deployed independently. A bug fix in the Notification Service does not require redeploying the Student or Exam services. This reduces deployment risk and downtime, which is critical during peak periods such as examination registration.

2.2 Fault Isolation

A failure in one service does not cascade to the entire system. If the Reporting Service crashes, students can still register for exams and view results. In a monolith, a single unhandled exception in a reporting query could bring the entire application down. The Exam Service handles Student Service unavailability gracefully by returning HTTP 502, rather than propagating the failure silently.

2.3 Independent Scalability

Services can be scaled horizontally based on their individual load profiles. During examination season, the Exam Service and Student Service receive high traffic; they can be scaled to multiple instances while the Reporting Service (low traffic) remains at one instance. This is more cost-effective than scaling an entire monolith.

2.4 Technology Flexibility

Each service can adopt the library or tooling best suited to its domain without affecting other services. The Transcript Service uses pdfkit for PDF generation; the Notification

Service uses kafkajs for event consumption. In a monolith, every team must agree on a single dependency version across the entire codebase.

2.5 Smaller, Focused Codebases

Each service has a single business responsibility, making it easier to understand, test, and maintain. A developer joining the Exam team only needs to understand Exam and Entry models, three routes, and one external HTTP call — not the entire university system.

2.6 Async Decoupling via Kafka

By publishing events (**student.created**, **exam.registered**, **result.published**) to a Kafka topic, the producing service is decoupled from its consumers. The Student Service does not need to know that a Notification Service exists — it simply emits an event. Adding a new consumer (e.g., an analytics service) requires no changes to existing services.

2.7 Alignment with Team Structure

Microservices allow future teams to own individual services end-to-end (Conway's Law). A student records team can own the Student Service; an assessments team can own Exam and Result services. This reduces coordination overhead across teams.

3. Risks and Mitigations

3.1 Distributed System Complexity

Risk: Microservices introduce network calls between services, making the system harder to reason about than a single in-process call. Failures can now be partial — a service may receive a response timeout rather than a clean error.

Mitigation: Strict HTTP status code contracts are defined in API Endpoint Design. The Exam Service wraps its Student Service call in a try/catch and maps errors to 502 Bad Gateway, giving callers a consistent error surface. All services expose a /health endpoint to support monitoring.

3.2 Data Consistency (No Distributed Transactions)

Risk: With one database per service, a business operation that spans multiple services (e.g., enrolling a student in an exam and updating their record simultaneously) cannot use a traditional ACID transaction across both databases.

Mitigation: The system uses eventual consistency via Kafka events for cross-service state changes. Synchronous REST calls are used only for read-verify operations (e.g., checking a student exists before creating an entry). Write operations that require consistency across services are modelled as sagas rather than distributed transactions.

3.3 Increased Infrastructure Overhead

Risk: Running eight services, eight MongoDB databases, NGINX, and Kafka/Kafka locally and in production is significantly more complex than deploying a single application binary.

Mitigation: Docker and docker-compose provide one-command startup for local development. Each service is containerised with a minimal node:20-alpine image. For the prototype, only Student and Exam services are active, reducing local resource usage substantially.

3.4 Inter-Service Latency

Risk: A synchronous REST call from Exam Service to Student Service adds network round-trip latency to every exam enrolment request. In a monolith, this would be an in-memory function call.

Mitigation: Both services run on the same Docker bridge network (erp-network), minimising latency to sub-millisecond for local and same-host deployments. Async Kafka events are used for operations that do not require an immediate response, avoiding synchronous chaining.

3.5 Harder to Debug and Trace

Risk: A single user request may touch multiple services. Tracing a failure across service logs without correlation IDs is time-consuming.

Mitigation: Each service logs the incoming request method and path. A correlation ID header (X-Request-Id) is propagated from NGINX through downstream service calls. For the prototype scope, structured console logging is used; production would adopt a centralised log aggregator (e.g., Loki or ELK).

3.6 Security Surface Area

Risk: Eight separately deployed services each expose HTTP endpoints, increasing the attack surface compared to a monolith with a single-entry point.

Mitigation: NGINX is the sole external entry point; service ports are not exposed to the public network in production (only NGINX port 80/443 is exposed). JWT tokens are validated independently by each service's own middleware using a shared secret, ensuring requests are authenticated even on internal network calls.

3.7 API Versioning / Backward Compatibility

Risk: Consumer services can break if a producer changes its API response structure or Kafka event payload schema. Because there is no compile-time type validation between services, changes such as renaming a field, removing a field, or altering a data type may go undetected until runtime — potentially in production. This risk is amplified in an event-driven system: a change to the **result.published** Kafka event payload could silently break the Notification Service without any immediate error being surfaced to the producer.

Mitigation: API versioning is not implemented in the current prototype. In a production system the following strategies would be applied:

Mitigation	Description
API versioning	Prefix all routes with /api/v1/. Breaking changes are released under /api/v2/ while consumers migrate.
Kafka schema versioning	Include a version field in every Kafka event payload (e.g. { "version": 1, "studentId": "..." }).
Schema Registry	Use Confluent Schema Registry with Avro or JSON Schema to enforce event contracts at publish time.
Contract testing	Use Pact (consumer-driven contract tests) to detect breaking API changes in CI before deployment.
Additive-only changes	During deprecation treat all changes as additive — only add new fields; never remove or rename existing ones.

4. Summary

The table below provides a concise risk-benefit summary for stakeholder review.

Dimension	Assessment	Net Verdict
Deployability	Independent per service	Benefit
Fault tolerance	Isolated failures; graceful degradation	Benefit
Scalability	Per-service horizontal scaling	Benefit
Consistency	Eventual (Kafka); requires saga pattern	Risk — mitigated
Infrastructure	Higher complexity; managed via Docker	Risk — mitigated
Latency	Network hops; minimised by co-location	Risk — mitigated
Observability	Harder tracing; correlation IDs applied	Risk — mitigated
Security	Larger surface; NGINX gateway + per-service JWT	Risk — mitigated
Codebase clarity	Smaller, focused services	Benefit
Team autonomy	Service ownership possible	Benefit
API versioning	No versioning in prototype; additive-change discipline and schema versioning recommended for production	Risk — to be mitigated